

Colour Modelling

James Foster

2026-09-09

Contents

Details	1
Introduction	2
Set up work space	2
Retina-level model	2
Photoreceptor Sensitivity	2
Response Function	3
Set Up Response Curve Data	5
Equal Absolute Sensitivity	5
Account for Sensitivity Difference	6
Photoreceptor Contrast	8
Colour Opponent Contrast Model	11
Set up functions	11
Simulate Bezold-Brücke Effect	14
COC Model LEDs	17

Details

DESCRIPTION

Models photoreceptor responses and spectral sensitivities using theoretical visual pigment templates. Simulates UV and green photoreceptor activation curves under experimental LED illumination to estimate relative photoreceptor excitation and the effects of unequal absolute sensitivity.

INPUTS

None (self-contained theoretical model)

OUTPUTS

GUV_colour_modelling.html (rendered report with figures)

REFERENCES

- Menzel, R., Ventura, D. F., Hertel, H., de Souza, J. M. and Greggers, U. (1986). Spectral sensitivity of photoreceptors in insect eyes. *J. Comp. Physiol. A* 158, 361–377.
- Stavenga, D. G. (2010). On visual pigment templates and the spectral shape of invertebrate rhodopsins and metarhodopsins. *J. Comp. Physiol. A* 196, 869–878.
- Kolyfetis, G. *et al.* (2025). Electrophysiological recordings reveal photoreceptor coupling in the dorsal rim areas of honeybee and bumblebee eyes. *Biol. Lett.* 21:20250234.<https://doi.org/10.1098/rsbl.2025.0234>
- Backhaus W. (1992) The Bezold-Brücke effect in the color vision system of the honeybee. *Vision Res.* 32(8):1425-31. doi: 10.1016/0042-6989(92)90199-s

Introduction

In this analysis we simulate the detection of UV and green stimuli of different intensities by honeybee photoreceptors, and consequently their perceived spectral contrast and colour. Two distinct approaches are applied:

1. Modelling of photoreceptor responses, based on their spectral sensitivity and intensity response curve.
2. Default COC colour modelling using `pavo`.

Set up work space

```
# Package for integration of functions
require("sfsmisc")

## Loading required package: sfsmisc

## Warning: package 'sfsmisc' was built under R version 4.5.2

# Package for colour modelling
require("pavo")

## Loading required package: pavo
```

Retina-level model

Photoreceptor Sensitivity

We define a function, `StavengaSpline()`, to generate visual pigment template curves for invertebrates. It supports A1 pigment types and calculates the α -, β -, and γ -absorption bands using modified log-normal models.

```
# Make a spline template for a visual pigment
StavengaSpline = function(spec_range = c(300, 700), #bounds of spectrum in nanometers
                           lambda_max, #peak sensitivity
                           a_type = 'a1') { #pigment type, only a1 available currently
  wlns = seq(from = min(spec_range),
```

```

        to = max(spec_range),
        length.out = 1e3) #
# modified lognormal
Mlognorm = function(wl,l_max, a0, a1)
{
  x = log10(wl/l_max)
  return(
    exp(-a0*x^2 *
        (1+a1*x+3*a1^2*x^2)
    )
  )
}
if(a_type == 'a1')
{
  #alpha band
  a_band = Mlognorm(wl = wlms,
                    l_max = lambda_max,
                    a0 = 380,
                    a1 = 6.09)

  #beta band
  b_band = 0.29*Mlognorm(wl = wlms,
                        l_max = 340,
                        a0 = 247,
                        a1 = 3.59)

  #gamma band
  g_band = 1.99*Mlognorm(wl = wlms,
                        l_max = 276,
                        a0 = 647,
                        a1 = 23.4)
}else
{stop('a2 and a3 pigments not yet implemented')}
# N.B. Stavenga normalises to max(a.band), I normalise to function max
r_stav = (a_band + b_band + g_band)/
  max(a_band + b_band + g_band)
return(  smooth.spline(x = wlms,
                      y = r_stav)  )
}#StavengaSpline <- function(spec.range, lambda.max)

```

Response Function

We model the photoreceptor response (in mV) to light intensity (photons/cm²/s) using a Naka-Rushton-like function. The parameters are based on honeybee photoreceptor recordings from Kolyfetis *et al.* (2025). See [1Data/250327_009_Apis_MainRetina_Kolyfetis2025.pdf](#). By default, we assume the semi-saturation point (k_{50}) is identical to that measured during those recordings. In a typical colour model, this is adjusted to the background average.

```

# Define a function for photoreceptor response
#params based on 250327_006 from Kolyfetis et al., 2025
PRresponse = function(
  quanta = 10^seq(from = 10,
                  to = 14,
                  length.out = 21), #photons/cm2/s
  vmax = 50, #mV

```

```

        k50 = 0.03*1e14, #inflection point
        n = 1.07) #slope
    {
mV = (vmax*quanta^n)/
      (k50^n + quanta^n)
    }
#this sets sensitivity in the following way
## rel_quanta = quanta/max(quanta)
## yy = mV/vmax
## sensitivity = k50*(yy/(1-yy))^(n-1)

```

We establish parameters for the response peaks of honeybee photoreceptors (Menzel et al., 1986) and the emission peaks for the experimental ultraviolet and green LEDs, using a 20nm bandwidth range for the LED spectral distribution.

```

# Response peak for honeybee photoreceptors Menzel et al., 1986
lmax_u = 343
lmax_g = 543
# Emission peak for each LED
led_u = 365
led_g = 528
# A 20nm (uniform) band for each
wlband = seq(from = -10,
              to = 10,
              by = 0.1)

```

Sensitivity Curve Predictions

We predict the photoreceptors' sensitivity curves across the insect-visible spectrum and specifically for the LED emission bands. We also calculate the integrated sensitivity for the green photoreceptor under green and UV LEDs to find its relative UV sensitivity (`g_rel_u`).

```

# Predict sensitivity curves across whole spectrum
u_photopig = predict(StavengaSpline(lambda_max = lmax_u))
g_photopig = predict(StavengaSpline(lambda_max = lmax_g))

# Specifically for the LEDs
#green photoreceptor
gPR_uLED = predict(StavengaSpline(lambda_max = lmax_g), x = led_u + wlband)
gPR_gLED = predict(StavengaSpline(lambda_max = lmax_g), x = led_g + wlband)
#UV photoreceptor
uPR_gLED = predict(StavengaSpline(lambda_max = lmax_u), x = led_g + wlband)
uPR_uLED = predict(StavengaSpline(lambda_max = lmax_u), x = led_u + wlband)

# integrated sensitivity
sens_gPR_uLED = integrate.xy(gPR_uLED)
sens_gPR_gLED = integrate.xy(gPR_gLED)
# spectral contrast (green-UV ratio)
g_rel_u = sens_gPR_uLED / sens_gPR_gLED

```

Set Up Response Curve Data

We define a range of UV intensities (photons/cm²/s) and compute simulated response curves for the UV and green photoreceptors under different absolute sensitivity assumptions.

```
xx = 10^seq(from = 11, to = 15, length.out = 1e3) #photons/cm2/s
example_g_equal = PRresponse(k50 = 3e12/g_rel_u, quanta = xx)
example_g_lows = PRresponse(k50 = 3e13/g_rel_u, quanta = xx)
example_u = PRresponse(k50 = 3e12, quanta = xx)

#set margins all around for individual plots
par(mar = c(4.5,4.5,4.5,4.5))
```

Equal Absolute Sensitivity

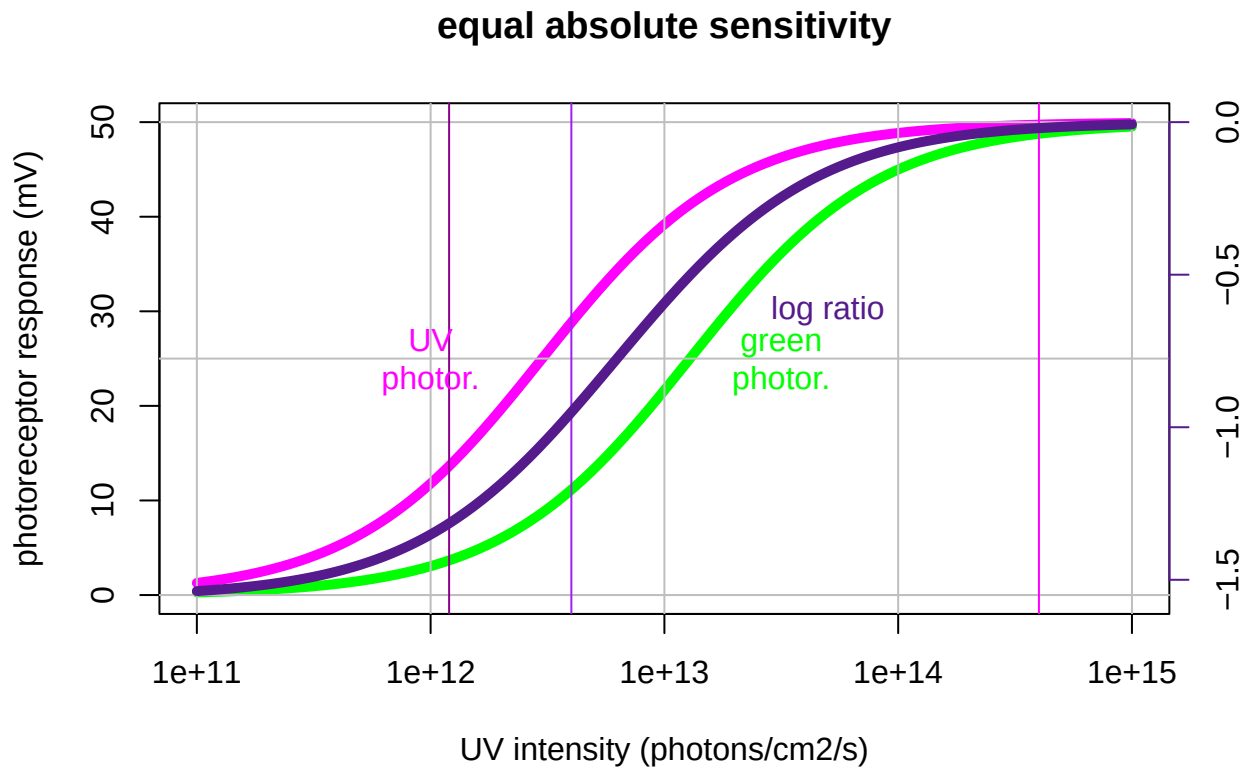
First, we plot simulated response curves assuming the green and UV photoreceptors have equal absolute sensitivity. We also compute and overlay the log ratio of green to UV photoreceptor responses.

```
#open plot with green photoreceptor
plot(x = xx,
     y = example_g_equal,
     type = 'l',
     col = 'green',
     lwd = 5,
     log = 'x',
     ylim = c(0, 50),
     main = 'equal absolute sensitivity',
     xlab = 'UV intensity (photons/cm2/s)',
     ylab = 'photoreceptor response (mV)')
#add UV photoreceptor
lines(x = xx,
     y = example_u,
     col = 'magenta',
     lwd = 5)
#add reference lines
abline(h = c(0,0.5,1.0)*50,
     v = 10^c(11:15),
     col = 'gray')
#add LEDs
abline(v = c(1.2e12, 4e12, 4e14),
     col = c('magenta4', 'purple', 'magenta'))
#label curves
text(x = 10^c(12, 13.5, 13.7),
     y = c(25, 25, 30),
     labels = c('UV\nphotor.',
                'green\nphotor.',
                'log ratio'),
     col = c('magenta', 'green', 'purple4')
     )
#add log ratio
#open new plot
par(new = TRUE)
plot(x = xx,
```

```

y = log(example_g_equal/example_u),
type = 'l',
ylim = c(-1.55, 0),
log = 'x',
lwd = 5,
col = 'purple4',
axes = FALSE,
xlab = '',
ylab = '')
#add axis
axis(side = 4,
     col = 'purple4')
#add axis label
mtext(side = 4,
      line = 2,
      text = 'log(green/UV) ',
      col = 'purple4')

```



Account for Sensitivity Difference

Next, we plot the response curves under a scenario where the green photoreceptor has a lower absolute sensitivity (semi-saturation constant k_{50} at a higher intensity) than the UV photoreceptor.

```

plot(x = xx,
     y = example_g_lows,

```

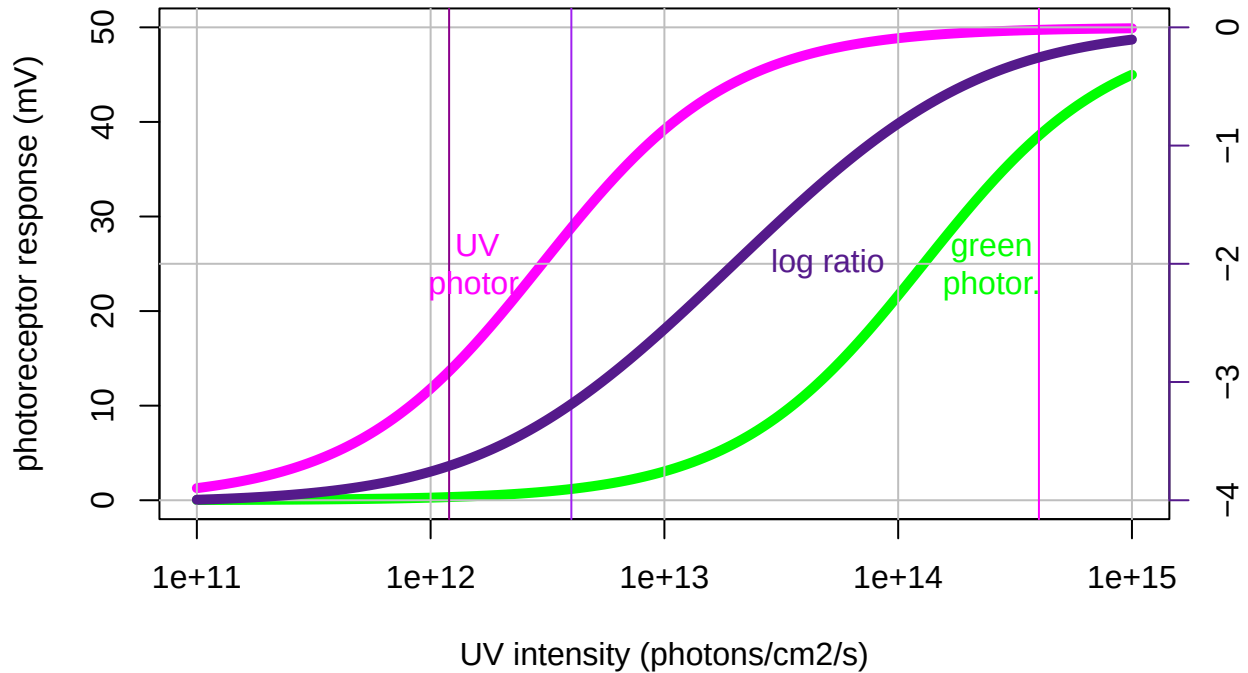
```

    type = 'l',
    col = 'green',
    lwd = 5,
    log = 'x',
    ylim = c(0, 50),
    main = 'green lower absolute sensitivity',
    xlab = 'UV intensity (photons/cm2/s)',
    ylab = 'photoreceptor response (mV)')
lines(x = xx,
      y = example_u,
      col = 'magenta',
      lwd = 5)
#add reference lines
abline(h = c(0,0.5,1.0)*50,
      v = 10^c(11:15),
      col = 'gray')
#add LEDs
abline(v = c(1.2e12, 4e12, 4e14),
      col = c('magenta4', 'purple', 'magenta'))
#label curves
text(x = 10^c(12.2, 14.4, 13.7),
     y = c(25, 25, 25),
     labels = c('UV\nphotor.',
                'green\nphotor.',
                'log ratio'),
     col = c('magenta', 'green', 'purple4'))
)
#add log ratio
#open new plot
par(new = TRUE)
plot(x = xx,
     y = log(example_g_lows/example_u),
     type = 'l',
     ylim = c(-4.0, 0),
     log = 'x',
     lwd = 5,
     col = 'purple4',
     axes = FALSE,
     xlab = '',
     ylab = '')

axis(side = 4,
     col = 'purple4')
mtext(side = 4,
     line = 2,
     text = 'log(green/UV)',
     col = 'purple4')

```

green lower absolute sensitivity



This relationship could also explain the change in perceived greenness of the bright UV stimulus, but would not explain the difference between dim and dimmer UV. Green receptors are more common in the honeybee retina than UV receptors ($\approx 4.4 : 1$), which may somewhat compensate for any difference in sensitivity.

Photoreceptor Contrast

Plot the simpler example to show how the green-opsin's β peak and saturation of the UV photoreceptor could contribute to a change in perceived green-ness between our dim and bright UV stimuli.

This combined two-panel figure presents:

1. The relative photoreceptor sensitivity profiles for both UV and green photoreceptors with integrated shaded areas for the UV and green LEDs.
2. The simulated response functions for both UV and green photoreceptors under the equal absolute sensitivity assumption, alongside the ratio of green to UV response on the right-hand axis.

```
par(mfrow = c(1,2),
    mar = c(5,5,0,3))
plot(u_photopig,
     type = 'l',
     lwd = 5,
     col = 'magenta',
     xlim = c(300, 700),
     ylim = c(0,1),
     xlab = 'wavelength (nm)',
```

```

        ylab = 'photoreceptor sensitivity\n(relative)',
    )
lines(g_photopig,
      lwd = 5,
      col = 'green')

with(uPR_uLED,
     polygon(x = c(x, rev(x)),
             y = c(y, rep(0, length(y))),
             col = adjustcolor('magenta',
                               alpha.f = 0.2),
             border = NA)
)
with(uPR_gLED,
     polygon(x = c(x, rev(x)),
             y = c(y, rep(0, length(y))),
             col = adjustcolor('magenta',
                               alpha.f = 0.2),
             border = NA)
)

with(gPR_uLED,
     polygon(x = c(x, rev(x)),
             y = c(y, rep(0, length(y))),
             col = adjustcolor('green',
                               alpha.f = 0.2),
             border = NA)
)
with(gPR_gLED,
     polygon(x = c(x, rev(x)),
             y = c(y, rep(0, length(y))),
             col = adjustcolor('green',
                               alpha.f = 0.2),
             border = NA)
)
abline(h = c(0,1.0),
       v = 100*c(3:7),
       col = 'gray')

plot(x = xx,
     y = example_g_equal,
     type = 'l',
     col = 'green',
     lwd = 5,
     log = 'x',
     ylim = c(0, 50),
     xlab = 'UV intensity (photons/cm2/s)',
     ylab = 'photoreceptor response (mV)')
lines(x = xx,
      y = example_u,
      col = 'magenta',
      lwd = 5)
#add reference lines

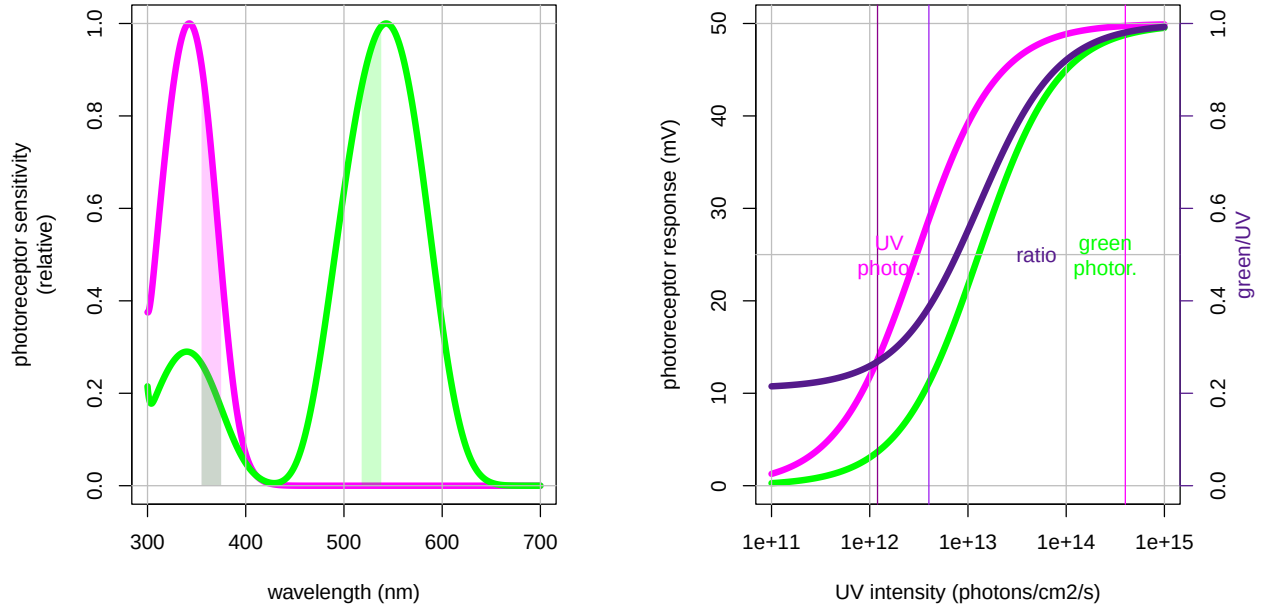
```

```

abline(h = c(0,0.5,1.0)*50,
       v = 10^c(11:15),
       col = 'gray')
#add LEDs
abline(v = c(1.2e12, 4e12, 4e14),
       col = c('magenta4', 'purple', 'magenta'))
#label curves
text(x = 10^c(12.2, 14.4, 13.7),
     y = c(25, 25, 25),
     labels = c('UV\nphotor.',
                'green\nphotor.',
                'ratio'),
     col = c('magenta', 'green', 'purple4'))
)
#add ratio
#open new plot
par(new = TRUE)
plot(x = xx,
     y = example_g_equal/example_u,
     type = 'l',
     ylim = c(0, 1),
     log = 'x',
     lwd = 5,
     col = 'purple4',
     axes = FALSE,
     xlab = '',
     ylab = '')

axis(side = 4,
     col = 'purple4')
mtext(side = 4,
      line = 2,
      text = 'green/UV',
      col = 'purple4')

```



Colour Opponent Contrast Model

The colour opponent contrast model (Backhaus, 1991) estimates colour perception in a two dimensional space, in which the two axes represent linear combinations of the responses of the UV-, blue- and green-sensitive photoreceptors, scaled such that the Manhattan distances between colour loci indicate the perceptual distance, and the point (0,0) corresponds to neutral grey.

The COC model simplifies the equation for photoreceptor responses from the Naka-Rushton (Naka & Rushton, 1966; Hill, 1910) equation,

$$R(P) = \frac{R_{\max} P^n}{P^n + K_{50}^n}$$

, to a hyperbolic equation,

$$R(P_{\text{relative}}) = \frac{P_{\text{relative}}}{P_{\text{relative}} + 1}$$

, where P_{relative} is the integrated photon catch of a photoreceptor divided by the photon catch for the background, the slope $n = 1$ (Backhaus & Menzel, 1987) is near constant around the inflection point at 50% saturation, and the 50% saturation parameter K_{50} adapts to the background intensity (Laughlin, 1989), which is set to $P = 1$ for simplicity.

Linear opponency between the responses of these photoreceptors is coded by just two colour channels,

$$A = -9.86 * R_{\text{UV}} + 7.70 * R_{\text{blue}} + 2.16 * R_{\text{green}}$$

$$B = -5.17 * R_{\text{UV}} + 20.25 * R_{\text{blue}} - 15.08 * R_{\text{green}}$$

, the weightings having been derived experimentally (Backhaus *et al.*, 1987). Perceived colour distances are the sum of differences in both A & B (city-block or Manhattan distance).

Set up functions

First set up a function that calculates the COC trajectories of monochromatic lights across a given range of relative intensities (measured relative to the background). We will also try to do this using `pavo` default functions, although these do not work in precisely the same way.

```

get_pavo_trajectories = function(test_wls, #wavelengths of monochromatic light
                                intensity, #relative intensities
                                background = rep(x = 1.0/sum(pavo::sensdata(visual = "apis")$wl<650),
                                times = length(pavo::sensdata(visual = "apis")$wl))
){
  # Establish the standard 300 to 700 nm wavelength backbone
  wl_backbone = pavo::sensdata(visual = "apis")$wl

  # Build the master pavo rspec matrix
  pavo_matrix = data.frame(wl = wl_backbone)

  for (target_wl in test_wls) {
    for (i in 1:length(intensity)) {
      col_name = paste0("WL_", target_wl, "_Mult_", i) # name to easily retrieve wavelength
      pavo_matrix[[col_name]] = ifelse(wl_backbone == target_wl,
                                      yes = intensity[i],
                                      no = 0)
    }
  }
  pavo_rspec = suppressMessages(as.rspec(pavo_matrix))

  # Run the native pavo vismodel pipeline
  pavo_vis = vismodel(
    rspecdata = pavo_rspec,
    visual = "apis",
    qcatch = "Ei",
    illum = "ideal",
    bkg = background,
    vonkries = TRUE, # Active automatic normalization
    relative = FALSE
  )

  # 5. Project directly into Color Opponent Coding (COC) Space
  pavo_coc = colspace(vismodeldata = pavo_vis,
                      space = "coc")

  # Build the matching metadata layout to map vectors back together
  metadata = expand.grid(
    intensity_step = 1:length(intensity),
    wavelength = test_wls
  )

  # Package everything into a clean data frame matching the Backhaus layout exactly
  trajectory_data = data.frame(
    wavelength = as.character(metadata$wavelength),
    intensity = intensity[metadata$intensity_step],
    x = pavo_coc$x,
    y = pavo_coc$y,
    chroma = with(pavo_coc, abs(x) + abs(y)), #absolute Manhattan distance from neutral grey
    uv = pavo_vis$s, #save the photoreceptor responses
    bl = pavo_vis$m,
    gr = pavo_vis$l
  )
}

```

```

    return(trajectory_data)
}

# plotting function
plot_coc_trajectories = function(trajectory_data, #data from get_backhaus_trajectories or get_pavo_traj
                                main_title = "Intensity-Dependent Colour Trajectories",
                                line_col = c("344" = "orchid4",
                                              "416" = "blue3",
                                              "494" = "cyan3",
                                              "532" = "darkgreen"), #needs to be named like trajectory
                                ref_point = 100 # indicate 1st point >100x background
                                ) {

  # 1dummy data frame structure for pavo canvas grid
  dummy_pavo = data.frame(s = 0,
                          m = 0,
                          l = 0,
                          x = c(-12, 12),
                          y = c(-12, 12))

  class(dummy_pavo) = c("colspace", "coc", "data.frame")

  # Open colour opponent contrast plot
  pavo::cocplot(dummy_pavo,
                main = main_title,
                col = "transparent")

  # Identify wavelengths
  unique_wls = unique(trajectory_data$wavelength)

  # Map over each wavelength vector to draw its expanding trajectory line
  for (wl_idx in unique_wls) {
    # Extract specific line color for this wavelength track
    current_col = if (wl_idx %in% names(line_col)){ line_col[wl_idx] }else{ "black" }

    # Draw continuous expanding color vector path
    with(trajectory_data[trajectory_data$wavelength == wl_idx, ],
        {
          lines(x = x,
                y = y,
                col = current_col,
                lwd = 3.5)
          # 1st point 10x brighter than background
          points(x = x[which(intensity > ref_point)][1],
                 y = y[which(intensity > ref_point)][1],
                 col = "black",
                 pch = 21,
                 bg = current_col,
                 cex = 1.3)
          # Indicate final position at maximum intensity
          points(x = x[length(x)],
                 y = y[length(y)],
                 col = "gray75",
                 pch = 21,

```

```

        bg = current_col,
        cex = 1.3,
        lwd = 3)
    }
)

}

Plot_COC_equal = function(x = 0,
                           y = 0,
                           threshold = 1.83,
                           ...)
{
  xx = c(x - threshold, # left corner
        0, # top
        x + threshold, #right corner
        0, # bottom
        x - threshold) #left corner
  yy = c(0,
        y + threshold,
        0,
        y - threshold,
        0)
  polygon(x = xx,
        y = yy,
        ...)
}

```

Simulate Bezold-Brücke Effect

In some experiments, honeybees interpreted coloured stimuli with the same reflectance spectrum as different colours depending on their brightness, although brightness differences did not play a role in stimulus discrimination. In the colour opponent contrast (COC) model (Backhaus, 1991), these shifts in perceived colour, known as the Bezold-Brücke effect in human colour vision (Purdy, 1931), could be predicted from the non-linear responses of photoreceptors to increasing brightness.

This effect can be demonstrated well at 344 nm (UV $\lambda_{\max}$), 416 nm and 494 nm (equal UV-B and B-G sensitivity) 532 nm (high G sensitivity).

```

test_wls = c(344, 416, 494, 532)
multipliers = 10^seq(-1, 3, length.out = 100) # 0.1x to 1000x contrast sweep

pavo_traj = get_pavo_trajectories(test_wls = test_wls,
                                intensity = multipliers
                                )

#plot together
# Split plotting grid into two window rows
par(mfrow = c(1, 1),
    mar = c(0,0,4,2))
line_colours = c("344" = "orchid4",

```

```

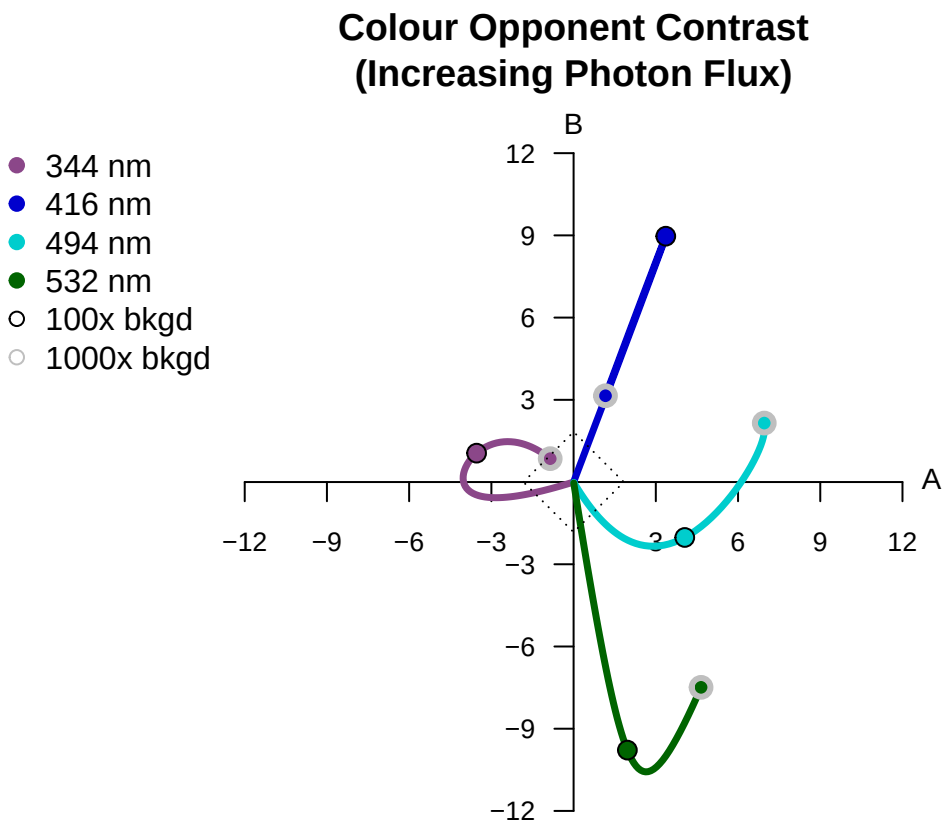
"416" = "blue3",
"494" = "cyan3",
"532" = "darkgreen")

plot_coc_trajectories(trajecory_data = pavo_traj,
                     main_title = "Colour Opponent Contrast\n(Increasing Photon Flux)",
                     line_col = line_colours
                     )

Plot_COC_equal(lty = 3)

legend(x = 'topleft',
       legend = c(paste(test_wls, 'nm'),
                  '100x bkgd',
                  paste0(round(max(multipliers)), 'x bkgd') ),
       col = c(line_colours,
               'black', 'gray75'),
       pch = c(rep(19, length(test_wls)),
               21, 21),
       bty = 'n')

```



The trajectories initially move away from the origin, and then turn back close to $10^2 \times$ background for blue and UV light.

We can also plot the chroma saturation and relative photoreceptor responses for each.

```
par(mfrow = c(1,2),
```

```

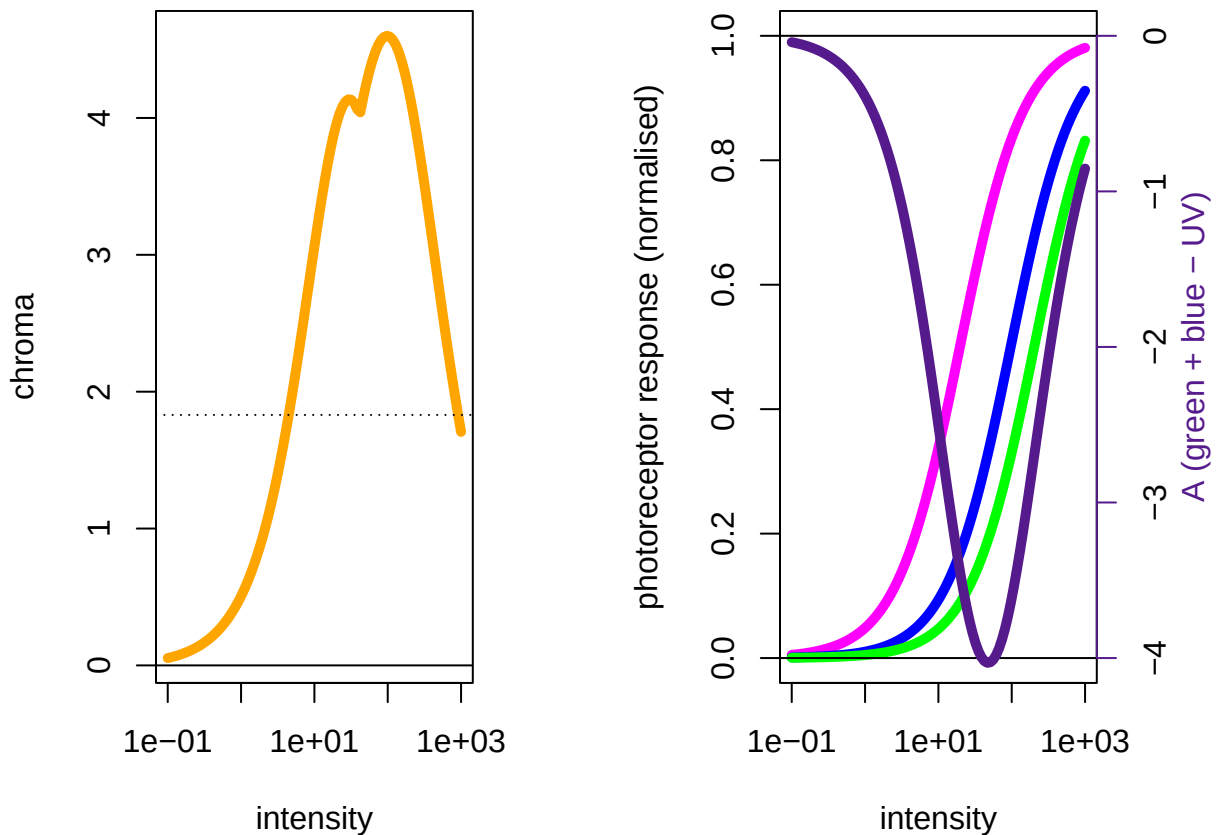
mar = c(5,5,0,3))

with(subset(pavo_traj,
           wavelength <400),
     {
       plot(x = intensity,
            y = chroma,
            log = 'x',
            type = 'l',
            col = 'orange',
            lwd = 5
            )
       abline(h = c(0, 1.83),
              lty = c(1, 3))
       plot(x = intensity,
            y = uv,
            log = 'x',
            type = 'l',
            col = 'magenta',
            lwd = 5,
            ylim = c(0,1),
            ylab = "photoreceptor response (normalised)"
            )
       lines(x = intensity,
            y = bl,
            col = 'blue',
            lwd = 5)
       lines(x = intensity,
            y = gr,
            col = 'green',
            lwd = 5)
       abline(h = c(0,1))
       #add ratio
       #open new plot
       par(new = TRUE)
       plot(x = intensity,
            y = x,
            type = 'l',
            ylim = c(-4, 0),
            log = 'x',
            lwd = 5,
            col = 'purple4',
            axes = FALSE,
            xlab = '',
            ylab = '')

       axis(side = 4,
            col = 'purple4')
       mtext(side = 4,
            line = 2,
            text = 'A (green + blue - UV)',
            col = 'purple4')
     }

```

)



Here we can see the drop in UV saturation at high intensities, and increasing responses of the blue and green photoreceptors. In terms of perception as UV, this peaks ($\min(A)$) close to $100\times$ background before returning towards perception as more long wavelength ($A \uparrow$) due to the greater rate of increase in blue- and green-sensitive photoreceptor responses.

COC Model LEDs

Assuming minimal reflections from the experimental room, the LEDs could plausibly be 3-5 log units brighter than the background. Assuming the bees are instead adapted to a bright afternoon sky, that range would be closer to -1.0 to 2.0 (see below).

```
study_wls = c(led_u, led_g)
study_multipliers = 10^seq(-1,
  3,
  by = 0.1) # 0.01x to 100000x contrast sweep

study_traj = get_pavo_trajectories(test_wls = study_wls,
  intensity = study_multipliers
)

#when does chromatic saturation peak?
print(
```

```

with(subset(study_traj, wavelength < 400),
     intensity[which(chroma == max(chroma))]),
digits = 2
)

## [1] 100

#what is chromatic saturation at maximum intensity?
print(
with(subset(study_traj, wavelength < 400 & intensity %in% c(100,1000) ),
     chroma),
digits = 2
)

## [1] 4.9 1.8

#plot together
# Split plotting grid into two window rows
par(mfrow = c(1, 1),
     mar = c(0,0,4,2))
study_colours = c('magenta',
                  'green')
names(study_colours) = study_wls

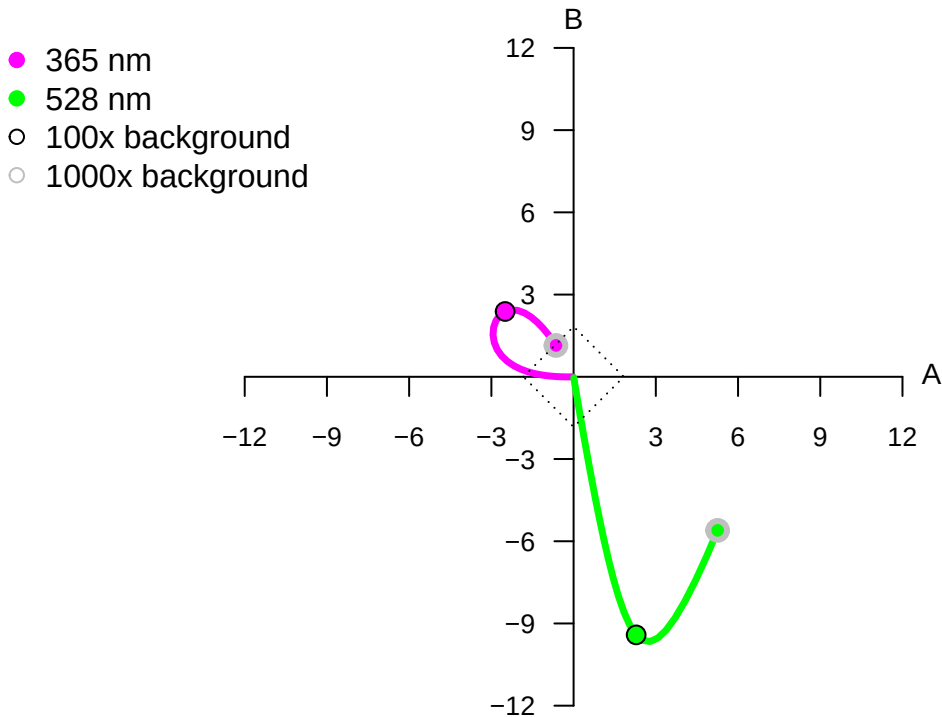
plot_coc_trajectories(trajjectory_data = study_traj,
                      main_title = "Green and UV LEDs\n(dark adapted)",
                      line_col = study_colours,
                      ref_point = 1e2
                      )

Plot_COC_equal(lty = 3)

legend(x = 'topleft',
       legend = c(paste(study_wls, 'nm'),
                  '100x background',
                  paste0(round(max(study_multipliers)), 'x background') ),
       col = c(study_colours,
               'black', 'gray75'),
       pch = c(rep(19,length(study_wls)),
               21, 21),
       bty = 'n')

```

Green and UV LEDs (dark adapted)



Here we do indeed observe a drop in UV saturation above 100 $\times$ background. At higher intensities, both the UV and green LEDs approach the “blue” quadrant in the positive A and B directions as the responses of the other photoreceptors (blue and green for UV; blue and UV for green) grow more rapidly than the saturated target receptor. These trajectories curve back towards the (0,0) achromatic (neutral grey) point without crossing the A or B axes, as all photoreceptors approach saturation. This occurs at lower intensities for the UV LED, which also stimulates the β peak of the other two receptors.

We could also assume light adaptation to a mid-afternoon blue sky with a total radiance of around 10^{14} photons/cm²/s.

```
sky_spectrum = within(sensdata(illum = "bluesky"),
                      {bluesky = (bluesky/sum(bluesky))*1e14}
)

light_set = c(dimmer = 1.2e12, dim = 4e12, bright = 4e14)

light_multipliers = c(seq(from = light_set[1],
                          to = light_set[2],
                          length.out = 50),
                      seq(from = light_set[2],
                          to = light_set[3],
                          length.out = 50)
)

light_traj = get_pavo_trajectories(test_wls = study_wls,
                                  intensity = light_multipliers,
```

```

                                background = sky_spectrum$bluesky
                                )

#what is chromatic saturation at each intensity?
print(
with(subset(light_traj, wavelength < 400 & intensity %in% light_set ),
  chroma),
digits = 2
)

## [1] 0.34 1.00 1.00 2.76

#plot together
# Split plotting grid into two window rows
par(mfrow = c(1, 1),
  mar = c(0,0,4,2))
study_colours = c('magenta',
  'green')
names(study_colours) = study_wls

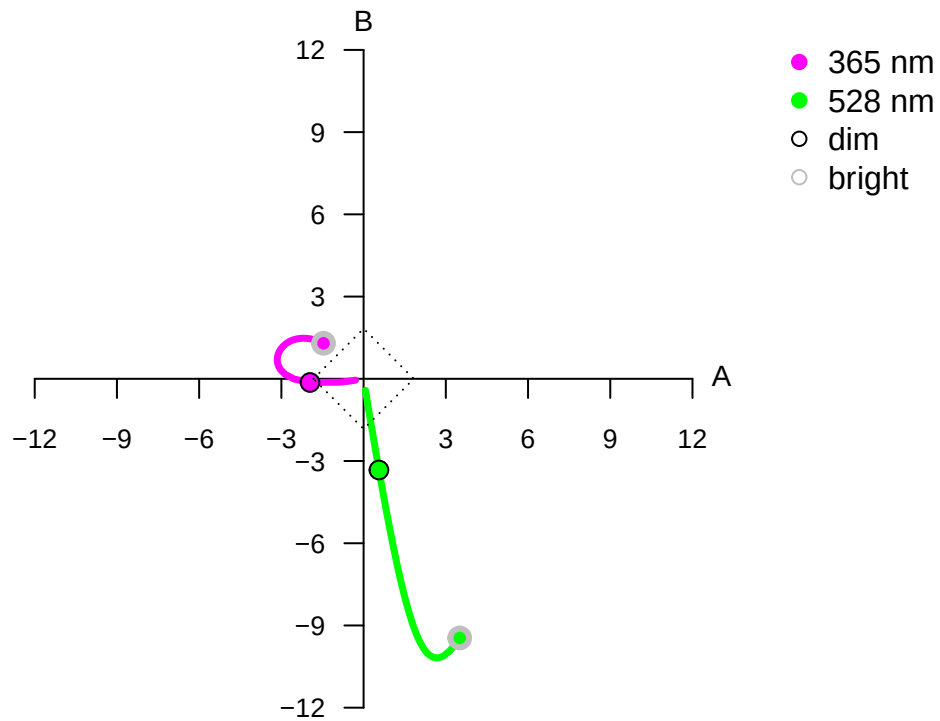
plot_coc_trajectories(trajjectory_data = light_traj,
  main_title = "Green and UV LEDs\n(light adapted)",
  line_col = study_colours,
  ref_point = light_set[2]
)

Plot_COC_equal(lty = 3)

legend(x = 'topright',
  legend = c(paste(study_wls, 'nm'),
    'dim',
    'bright'),
  col = c(study_colours,
    'black', 'gray75'),
  pch = c(rep(19,length(study_wls)),
    21, 21),
  bty = 'n')

```

Green and UV LEDs (light adapted)



In the light adapted situation, the Bezold-Brücke effect is more limited, but still occurs for the brighter stimuli. The dim stimuli are dimmer than the background, resulting in activation of only their target photoreceptor (UV $A < 0$, green $B < 0$) but very little activation of the blue-sensitive photoreceptor ($A \& B > 0$).